Regression Model

When examining housing unaffordability across Charlotte, a regression framework was used to model housing cost burden as a continuous outcome. The modeling process shows a structured pipeline, beginning with data loading from a database of 458 observations and 23 features, followed by custom train/test splits. This then led to model refinement. The goal of this analysis was not only to achieve strong predictive performance, but also to better understand how different neighborhood characteristics contribute to variations in housing burden.

Multiple Linear Regression

The process began with a Multiple Linear Regression (OLS) model to establish a baseline. A wide range of features was included initially, covering socioeconomic conditions, housing characteristics, demographic factors, and measures of accessibility. Prior to modeling, the features with high multicollinearity were removed during the explanatory data analysis (EDA). To refine the model, Recursive Feature Elimination (RFE) was used to identify the most relevant predictors using cross-validation and variables exhibiting high multicollinearity were removed to improve stability. The RFE results offered that model performance, also referred to as adjusted R^2, was best around $k = 4$ features. These features specifically being household_income, home_ownership, food_nutrition, and age_of_residents.

Although the optimal subset of predictors contained four features, a 7-feature model was initially chosen to capture the broader neighborhood dynamics. These features included new_constructions, transit_proximity, and grocery_proximity. This decision was supported by having nearly identical adjusted R^2 values of 0.719 versus 0.718. This suggests minimal performance reduction while gaining interpretability.

The OLS model was fitted using statsmodels and its performance was evaluated using R^2 and RMSE on both training and test sets. With this, the model achieved a test R^2 of approximately 0.69 with an RMSE near 0.089 and a train R^2 of about 0.73 with an RMSE of 0.0854. While these results showed moderate predictive power, the statistical results showed that both transit_proximity and grocery_proximity were not significant predictors as their p-values were greater than 0.05, suggesting that these variables more so added noise rather than meaningful insight to the explanatory value.

Diagnostics

Taking it a step further to assess model validity, the residual diagnostics were generated in coding, including different plots such as histograms, Q-Q plots, and residuals vs fitted plots. These outputs revealed two key violations of linear regression assumptions; First non-linearity, showing U-shaped residual pattern with LOWESS smoothing and second, heteroscedasticity, a fan-shaped spread of residuals. Additionally, five extreme outliers were identified when using a 3-standard-deviation threshold. Investigation of these observations showed that they correspond to neighborhoods with unusually high housing burden, which was relative to income, confirming they are real phenomena rather than error in data.

Given these limitations, alternative modeling strategies were explored. Polynomial regression models, with degrees 2-4, were considered to introduce non-linearity. These models implemented the usage of a pipeline (PolynomialFeatures and LinearRegression). The model comparison showed when using degree 2, an R^2 of approximately 0.83 and an RMSE of around 0.067 were achieved and when using degree 3 and 4, severe overfitting and poor generalization

occurred. Although the degree 2 model significantly improved the predictive performance, its interpretation was limited due to complex feature interactions.

Generalized Additive Model

A more flexible approach was then implemented using a Generalized Additive Model (GAM) using spline features for each predictor, which allowed to balance flexibility and interpretability. The GAM was trained using grid search. This was to optimize the smoothing parameters. With the 7-feature set the model achieved an R^2 of about 0.83 and an RMSE of around 0.0668. The diagnostic plots generated in the code delivered clear improvements over OLS, which involved more symmetric residual distribution and reduced heteroscedasticity.

Final Model Selection

More analysis using partial dependencies revealed that the new_constructions, transit_proximity, and grocery_proximity all contributed minimal explanatory power. These features then were removed and the final model was retrained on the four key predictors identified earlier. The final 4-feature GAM produced the strongest results of a test R^2 of ~0.83 and a test RMSE of about 0.66. To evaluate generalization, a 5-fold cross validation was performed on the full dataset which then resulted in a mean R^2 of 0.8059 ± 0.0815 and a mean RMSE of 0.0713 ± 0.0181. These results were consistent with test performance. This suggests that the model generalizes well despite some variability due to extreme outliers.

Overall, the modeling process demonstrated a progression from a simple linear approach to a much more flexible, accurate, non-linear model. The final GAM not only improves predictive performance but also captures complex, non-linear relationships between

neighborhood characteristics and housing cost burden. This made it the most appropriate model for this analysis.

Appendix A: Model Implementation Code

This appendix contains python code used to build, evaluate, and select the final regression model described in the report.

```
# Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sqlite3 as db
import importlib
import sys
if 'trainTestSplit' in sys.modules:
   del sys.modules['trainTestSplit'] #remove cached version of trainTestSplit if it
exists
sys.path.append("../src")
from trainTestSplit import get_reg_splits, modified_reg_splits, gam_splits
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, root_mean_squared_error, PredictionErrorDisplay
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score
from sklearn.feature_selection import RFE
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import Pipeline
import scipy.stats as st
import statsmodels.api as sm
import pylab as py
from pygam import LinearGAM, s

# Load data
conn = db.connect("../data/charlotte_housing.db")
master_df = pd.read_sql_query("SELECT * FROM master", conn)
conn.close()

X_train, X_test, y_train, y_test = get_reg_splits(master_df)

#Recursive Feature Elimination
lr = LinearRegression()
```

```
for k in range(1, X_train.shape[1] + 1):
   rfe = RFE(estimator=lr, n_features_to_select=k)
   rfe.fit(X_train, y_train)
   X_selected = rfe.transform(X_train)
   r2_scores = cross_val_score(lr, X_selected, y_train, cv=8, scoring="r2")
   mean_r2 = r2_scores.mean()
   n, p = X_train.shape[0], k
   adj_r2 = 1 - (1 - mean_r2) * (n - 1) / (n - p - 1)

   selected = X_train.columns[rfe.support_].tolist()
   print(f"\nk={k}: Adj R²={adj_r2:.4f}")

# Modified Feature Set(7 features)
X_train, X_test, y_train, y_test = modified_reg_splits(master_df)

# OLS Regression Model
X2_train = sm.add_constant(X_train)
ols_model = sm.OLS(y_train, X2_train).fit()
X2_test = sm.add_constant(X_test)
y_pred = ols_model.predict(X2_test)


r2_train     = r2_score(y_train, y_pred_train)
rmse_train   = root_mean_squared_error(y_train, y_pred_train)
r2_test      = r2_score(y_test, y_pred)
rmse_test    = root_mean_squared_error(y_test, y_pred)

print(f"OLS Model Performance: Housing Cost Burden")
print(f"{'R²':<20} {r2_train:>15.4f} {r2_test:>15.4f}")
print(f"{'RMSE':<20} {rmse_train:>15.4f} {rmse_test:>15.4f}")

# Polynomial Regression Models
results = {}
for degree in [2, 3, 4]:
   poly_pipeline = Pipeline([
       ('poly', PolynomialFeatures(degree=degree, include_bias=False)),
       ('model', LinearRegression())
   ])
poly_pipeline.fit(X_train, y_train)
   y_pred_poly = poly_pipeline.predict(X_test)
   r2 = r2_score(y_test, y_pred_poly)
```

```
    rmse = root_mean_squared_error(y_test, y_pred_poly)
results[degree] = {"R²": r2, "RMSE": rmse}

# Generalized Additive Model (7 features)
gam = LinearGAM(s(0) + s(1) + s(2) + s(3) + s(4) + s(5) + s(6))
gam.gridsearch(X_train.values, y_train.values)
y_pred_gam = gam.predict(X_test.values)
print("GAM vs OLS Performance on Test Set:")
print(f"GAM   — R²: {r2_gam:.4f} | RMSE: {rmse_gam:.4f}")
print(f"OLS   — R²: {r2_score(y_test, y_pred):.4f} | RMSE:
{root_mean_squared_error(y_test, y_pred):.4f}")

#Final Model (4 features)
X_train, X_test, y_train, y_test = gam_splits(master_df)
gam = LinearGAM(s(0) + s(1) + s(2) + s(3))
gam.gridsearch(X_train.values, y_train.values)
y_pred_gam = gam.predict(X_test.values)

print("GAM vs OLS Performance on Test Set:")
print(f"GAM   — R²: {r2_gam:.4f} | RMSE: {rmse_gam:.4f}")
print(f"OLS   — R²: {r2_score(y_test, y_pred):.4f} | RMSE:
{root_mean_squared_error(y_test, y_pred):.4f}")

#Cross Validation
X_all = pd.concat([X_train, X_test]).values
y_all = pd.concat([y_train, y_test]).values
kf = KFold(n_splits=5, shuffle=True, random_state=42)
cv_r2, cv_rmse = [], []
for fold, (train_idx, val_idx) in enumerate(kf.split(X_all)):
    X_fold_train, X_fold_val = X_all[train_idx], X_all[val_idx]
    y_fold_train, y_fold_val = y_all[train_idx], y_all[val_idx]
    gam_fold = LinearGAM(s(0) + s(1) + s(2) + s(3))
    gam_fold.gridsearch(X_fold_train, y_fold_train)
    y_fold_pred = gam_fold.predict(X_fold_val)
    fold_r2   = r2_score(y_fold_val, y_fold_pred)
    fold_rmse = root_mean_squared_error(y_fold_val, y_fold_pred)
    cv_r2.append(fold_r2)
    cv_rmse.append(fold_rmse)
print(f"\nMean R²:  {np.mean(cv_r2):.4f} ± {np.std(cv_r2):.4f}")
print(f"Mean RMSE: {np.mean(cv_rmse):.4f} ± {np.std(cv_rmse):.4f}")
```